PROGRAMACIÓN ORIENTADA A OBJETOS

Facultad de Ingeniería – UNER | 2026

Laboratorio de curvas

Generación, transformación, visualización y análisis numérico

Informe técnico del Trabajo Práctico

Título del proyecto:	*Laboratorio de curvas*
Integrante/s:	*Durand, Camila Ayelen*
	Sierak Naiara
Fecha de entrega:	*13/11/2026*
Repositorio:	*https://github.com/Camila20197/laboratorio_curvas*
Docente/s:	*Javier E. Diaz Zamboni, Jordán F. Insfrán, Juan Rizzato, Feliciano Franco*

Parte A – Preguntas

1. Recursividad

a. ¿Qué es la recursividad?

Una **función recursiva** es una función que se llama a sí misma. La recursión puede ser **directa**, cuando la función se invoca explícitamente dentro de su propio código, o **indirecta**, cuando la función llama a otra que, a su vez, termina invocando a la primera.

En tiempo de ejecución, cada llamada reserva espacio para un conjunto completo de variables propias mediante asignación dinámica de memoria: el **registro de activación**, que aloja los parámetros y las variables locales de esa llamada. Los registros se apilan en la **pila del sistema** a medida que las funciones se invocan y se eliminan cuando cada una finaliza; por eso la pila crece y decrece constantemente y coexisten variables con valores presentes y pendientes.

La recursividad se usa principalmente con dos estrategias: **dividir para vencer** (dividir la entrada en subproblemas, resolverlos recursivamente y combinar sus soluciones; por ejemplo, Torres de Hanoi, búsqueda binaria, quicksort o la curva de Koch de este trabajo) y **búsqueda exhaustiva con rastreo inverso** (por ejemplo, el salto del caballo). Iteración y recursión son equivalentes en el sentido de que permiten resolver el mismo tipo de problemas.

b. Condiciones para que se ejecute correctamente en un tiempo finito

1. **Caso base (condición de salida).** Debe existir al menos un caso en el que la función devuelva un resultado directamente, sin volver a llamarse.
2. **Avance hacia el caso base.** Cada llamada recursiva debe trabajar sobre un problema más pequeño que el original (por ejemplo, n − 1 o la mitad del intervalo), de modo que la secuencia de llamadas alcance el caso base en una cantidad finita de pasos.
3. **Profundidad de recursión finita y pequeña.** Cada nivel ocupa un registro de activación en la pila; si la profundidad es excesiva se agota la memoria de la pila (en Python se lanza RecursionError al superar el límite de recursión, que por defecto es 1000). También debe controlarse la cantidad total de llamadas: en este trabajo la curva de Koch limita el nivel a N = 6 porque cada nivel multiplica por 4 la cantidad de segmentos.

c. Ejemplo concreto: Torres de Hanoi

Hay tres torres y N discos apilados en la torre de origen, de mayor a menor. Hay que pasarlos a la torre de destino moviendo un disco por vez y sin colocar nunca un disco grande sobre uno más chico. El ejemplo proviene del material de la asignatura:

```
def mover_torre(altura, origen, destino, intermedio):
    if altura >= 1:                                  # caso recursivo
        mover_torre(altura - 1, origen, intermedio, destino)
        mover_disco(origen, destino)
        mover_torre(altura - 1, intermedio, destino, origen)
    # si altura == 0 no hace nada: caso base

def mover_disco(desde, hacia):
    print("mover disco de", desde, "a", hacia)

if __name__ == "__main__":
    mover_torre(3, "A", "B", "C")
```

Funcionamiento. Para mover una torre de altura N desde el origen al destino: (1) se mueven recursivamente los N − 1 discos superiores a la torre intermedia; (2) se mueve el disco más grande al destino; (3) se mueven recursivamente los N − 1 discos desde la torre intermedia al destino. Se cumplen las tres condiciones del punto b: el caso base es altura = 0 (no hay nada que mover), cada llamada reduce la altura en 1 y la profundidad máxima es N.

Traza para N = 2 (mover_torre(2, "A", "B", "C")): la primera llamada interna mueve 1 disco de A a C (imprime «A → C»); luego se mueve el disco grande («A → B»); por último la segunda llamada interna mueve el disco de C a B («C → B»). Son 3 movimientos. En general, la cantidad de movimientos cumple $H(n) = 2 \cdot H(n-1) + 1 = 2^n - 1$, por lo que la complejidad es $O(2^n)$.

2. Coordenadas homogéneas

a. Qué son y cómo se relacionan con el pipeline de graficación

En coordenadas homogéneas un punto del plano (x, y) se representa con un vector de tres componentes (x, y, w) con $w \neq 0$; el punto cartesiano correspondiente es (x/w, y/w). Normalmente se usa w = 1, es decir (x, y, 1). Un vector es homogéneo si al menos uno de sus elementos es distinto de cero.

Su utilidad es que **primitivas y transformaciones se representan de forma unificada**: las primitivas como vértices (vectores) y las transformaciones como matrices. Con dos coordenadas, la escala y la rotación son productos de matrices 2×2, pero la traslación es una suma y no puede escribirse como una matriz 2×2. Al agregar la tercera coordenada, **escala, rotación y traslación pasan a ser todas productos por matrices 3×3**, y aplicar varias transformaciones seguidas equivale a multiplicar sus matrices.

Esto es lo que aprovecha el **pipeline de graficación** con el esquema de renderizado MVP (modelo, vista, proyección): la matriz de modelo M ubica y transforma los vértices del objeto, la matriz de vista V ubica al observador y la matriz de proyección P determina los planos de recorte y cómo se ve la figura final. Todos los vértices v se procesan con una única operación matricial:

$$(P \times V \times M) \times v = v'$$

Finalmente, la transformación de puerto de visión (viewport) determina qué parte de la superficie de dibujo de la ventana se usa para mostrar el gráfico. En este trabajo, la clase Transformacion aplica exactamente esta idea: cada transformación concreta define su matriz 3×3 y el método aplicar_punto calcula $v' = M \cdot v$ sobre el vector homogéneo del punto.

b. Ejemplos en 2D de cada transformación de modelo-vista

Los resultados de estos ejemplos se verificaron con las clases Escalado, Rotacion y Traslacion del proyecto (los ángulos se expresan en radianes).

Escala. Multiplica cada coordenada por un factor (sx, sy). Punto original P = (2, 3), con sx = 2 y sy = 3:

$$\begin{vmatrix} 2 & 0 & 0 \\ 0 & 3 & 0 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} 2 \\ 3 \\ 1 \end{vmatrix} = \begin{vmatrix} 2 \cdot 2 \\ 3 \cdot 3 \\ 1 \end{vmatrix} = \begin{vmatrix} 4 \\ 9 \\ 1 \end{vmatrix}$$

El punto se transforma en P′ = (4, 9).

Rotación. Gira el punto un ángulo θ alrededor del origen. Punto original P = (1, 0), con θ = 90° = π/2 rad (cos θ = 0, sen θ = 1):

$$\begin{vmatrix} \cos\theta & -\text{sen}\,\theta & 0 \end{vmatrix} \quad \begin{vmatrix} 1 \end{vmatrix} \quad \begin{vmatrix} 0 & -1 & 0 \end{vmatrix} \quad \begin{vmatrix} 1 \end{vmatrix} \quad \begin{vmatrix} 0 \end{vmatrix}$$

$$\begin{vmatrix} \text{sen}\,\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} 0 \\ 1 \end{vmatrix} = \begin{vmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} 0 \\ 1 \end{vmatrix} = \begin{vmatrix} 1 \\ 1 \end{vmatrix}$$

El punto se transforma en P′ = (0, 1): el punto del eje x pasa al eje y.

Traslación. Desplaza el punto (dx, dy). Solo puede escribirse como producto matricial gracias a la coordenada homogénea. Punto original P = (3, 4), con dx = 5 y dy = −2:

$$\begin{vmatrix} 1 & 0 & 5 \\ 0 & 1 & -2 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} 3 \\ 4 \\ 1 \end{vmatrix} = \begin{vmatrix} 3+5 \\ 4-2 \\ 1 \end{vmatrix} = \begin{vmatrix} 8 \\ 2 \\ 1 \end{vmatrix}$$

El punto se transforma en P′ = (8, 2).

Composición. Como todas son matrices, varias transformaciones se combinan multiplicándolas; la que se aplica primero queda a la derecha. Para rotar 90° y luego trasladar (5, −2) el punto (1, 0): M = T · R, y T · R · (1, 0, 1) = T · (0, 1, 1) = (5, −1, 1). El orden importa: R · T aplicada al mismo punto da (2, 6), porque primero traslada a (6, −2) y después rota.

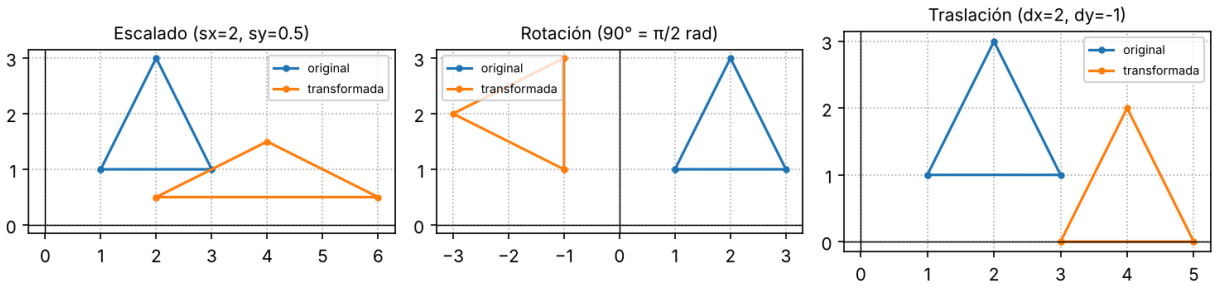


Figura 1. Escalado, rotación y traslación aplicados a un triángulo con las clases del proyecto.

3. Integración numérica

a. Método del trapecio

El método del trapecio pertenece a las fórmulas de integración de **Newton-Cotes**: se interpola la función con un polinomio y se integra el polinomio en lugar de la función. En el trapecio el polinomio es de **orden 1**, es decir, la recta que une (a, f(a)) con (b, f(b)); el área bajo esa recta es la de un trapecio:

$$\int_a^b f(x)\,dx \approx (b-a)\cdot\left[\,f(a)+f(b)\,\right]/2$$

Para mejorar la aproximación se usa la **regla del trapecio extendida (compuesta)**: se divide [a, b] en n subintervalos de ancho h = (b − a)/n, con nodos $x_i = a + i\cdot h$, y se suman las áreas de los n trapecios. Los puntos interiores aparecen dos veces porque cada uno es lado de dos trapecios vecinos:

$$\int_a^b f(x)\,dx \approx (h/2)\cdot\left[\,f(x_0)+2\cdot f(x_1)+\ldots+2\cdot f(x_{n-1})+f(x_n)\,\right]$$

Por ejemplo, para $\int_0^1 x^2\,dx = 1/3$: con n = 1 se obtiene 0,5; con n = 2, 0,375; con n = 4, 0,34375.

b. Expresión general del error de aproximación

Para un solo trapecio, el error de truncamiento es:

$$E_n = -(1/12)\cdot f''(\xi)\cdot(b-a)^3, \quad \text{con } \xi \in (a, b)$$

Para la regla extendida con n subintervalos, sumando el error de cada trapecio y reemplazando la segunda derivada por su valor medio $\bar{f}''$ en [a, b]:

$$E_a = -[(b - a)^3 / (12 \cdot n^2)] \cdot f'' = -[(b - a) / 12] \cdot h^2 \cdot f''$$

c. Significado e importancia de la expresión del error

- **La segunda derivada mide la curvatura.** El error es proporcional a f″: cuanto más se curva la función, más se aleja de los segmentos rectos. Si la función es lineal (f″ = 0) el método es **exacto**.
- **El signo indica si se sobreestima o subestima.** Si la función es cóncava hacia arriba (f″ > 0) el trapecio queda por encima de la curva y la regla **sobreestima** la integral (E < 0); si es cóncava hacia abajo, la subestima.
- **El error es de orden h².** Al duplicar la cantidad de subintervalos, h se reduce a la mitad y el error se divide aproximadamente por 4. La Tabla 1, obtenida con la clase Trapecio del proyecto para $\int_0^1 x^2\,dx$, lo confirma (como f″ = 2 es constante, el cociente es exactamente 4).

n	h	Aproximación	Error (aprox. − 1/3)	Error anterior / error actual
1	1	0,500000	0,166667	—
2	0,5	0,375000	0,041667	4,0
4	0,25	0,343750	0,010417	4,0
8	0,125	0,335938	0,002604	4,0
16	0,0625	0,333984	0,000651	4,0

Tabla 1. Convergencia del método del trapecio para $\int_0^1 x^2\,dx$ (valor exacto 1/3).

Importancia. La expresión permite (1) estimar la magnitud del error sin conocer el valor exacto de la integral, (2) **elegir de antemano cuántos subintervalos se necesitan** para una tolerancia dada, despejando $n \geq \sqrt{[(b - a)^3 \cdot \text{máx}|f''| / (12 \cdot \text{tolerancia})]}$, y (3) comparar métodos según su orden de convergencia (por ejemplo, ⅓ de Simpson es de orden h^4). Ejemplo: para $\int_0^{\pi} \text{sen}(x)\,dx$ (máx|f″| = 1) y una tolerancia de 10^{-4} se necesitan n ≥ 161 subintervalos. Con n = 100 la cota es $2{,}6 \cdot 10^{-4}$ y el error real obtenido con el programa es $1{,}6 \cdot 10^{-4}$, dentro de la cota.

Parte B – Explorador gráfico de curvas y algoritmos numéricos

i. Descripción del problema y de la solución

Se pide una **aplicación de terminal orientada a objetos** que permita trabajar con curvas en el plano cartesiano mediante un menú de opciones sencillo y una ventana de visualización. Los requerimientos funcionales son:

- **RF1 – Generación de curvas** a partir de funciones y = f(x) polinómica, seno y exponencial en un intervalo [a, b] con una cantidad de puntos elegida por el usuario.
- **RF2 – Visualización** con matplotlib: título, leyendas, ejes con marcadores y etiquetas, la curva, los extremos del intervalo de integración, la raíz encontrada, actualización tras cada operación y ajuste del intervalo visible.
- **RF3 – Curva de Koch** generada con un algoritmo recursivo, como mínimo hasta la sexta iteración.
- **RF4 – Búsqueda de raíces** por el método de bisección, con tolerancia o número máximo de iteraciones.
- **RF5 – Integración** por el método de los trapecios, separada de la interfaz y de la representación gráfica.

Además se requieren transformaciones geométricas de escalado, rotación y traslación, persistencia de las curvas, pruebas unitarias con cobertura ≥ 60 % y un diseño orientado a objetos documentado con UML.

Solución desarrollada. El sistema se organizó en paquetes dentro de modules/, cada uno con una responsabilidad única y una clase por archivo: funciones matemáticas, geometría con Punto y Curva, generadores de curvas a partir de funciones y de Koch, transformaciones en coordenadas homogéneas, algoritmos numéricos con sus resultados, visualización y persistencia en JSON. Las jerarquías abstractas de FuncionMatematica, Transformacion, AlgoritmoNumerico y Resultado permiten usar las clases concretas de forma polimórfica y extender el sistema sin modificar el código existente.

PENDIENTE: la clase MenuConsola (interfaz de terminal) y el punto de entrada main.py, que integran todos los paquetes. Completar esta sección con una descripción del menú cuando esté terminado.

ii. Diagrama de clases UML

El diagrama sigue las convenciones de la asignatura: «abstract class» y «concrete class» sobre el nombre; métodos abstractos en cursiva y con asterisco; «get» para las propiedades de solo lectura; atributos y métodos de clase subrayados; - privado, # protegido y + público. Relaciones: herencia con triángulo hueco, composición con rombo relleno, agregación rombo vacío, asociación línea simple y dependencia de línea punteada con flecha abierta. El archivo editable se encuentra en docs/diagrama_clases.drawio.

Enlace al diagrama:
https://drive.google.com/file/d/12-sP4E2VLtdzPCEbA21vclz0mfXpq0wH/view?usp=sharing

PENDIENTE: actualizar el diagrama con la versión final de MenuConsola y verificar que los nombres coincidan con el código final

iii. Justificación de la organización en módulos y clases

Paquete	Clases	Responsabilidad
funciones	FuncionMatematica (abstracta), FuncionPolinomica, FuncionSeno, FuncionExponencial	Representar una función y evaluarla en un punto (RF1).
geometria	Punto, Curva	Punto: coordenadas y conversión a homogéneas. Curva: conjunto ordenado de puntos, longitud y recorrido.
generadores	GeneradorCurva, GeneradorKoch	Crear curvas: muestreando una función (RF1) o de forma recursiva (RF3).
transformaciones	Transformacion (abstracta), Escalado, Rotacion, Traslacion	Aplicar transformaciones de modelo-vista con matrices homogéneas 3×3.
algoritmos	AlgoritmoNumerico (abstracta), Biseccion, Trapecio, SinCambioDeSignoError	Cálculo numérico sobre una función en [a, b] (RF4 y RF5).
resultados	Resultado (abstracta), ResultadoBiseccion, ResultadoIntegracion	Conocer los valores obtenidos por un algoritmo y describirlos como texto.
visualizacion	Visualizador	Única clase que usa matplotlib (RF2).
persistencia	RepositorioCurvas	Guardar, cargar y listar curvas en archivos JSON.

Tabla 2. Paquetes del proyecto y sus responsabilidades.

Principios de diseño aplicados

- **Responsabilidad única (SRP).** Curva no sabe guardarse lo hace RepositorioCurvas, los algoritmos no dibujan ni leen la consola ya que lo exige RF5 y Resultado no calcula ni imprime: su método resumen() devuelve texto y la interfaz decide qué hacer con él.
- **Abierto/cerrado (OCP) y polimorfismo.** Las cuatro jerarquías abstractas permiten agregar una función, transformación o algoritmo nuevo, por ejemplo: Newton-Raphson o ⅓ de Simpson creando una subclase, sin modificar el código existente. AlgoritmoNumerico aplica el **patrón Estrategia**: los algoritmos son intercambiables detrás de calcular(funcion, a, b).
- **Sustitución de Liskov (LSP).** Bisección agrega una precondición propia, cambio de signo. Para no romper el contrato del padre, este declara que calcular puede lanzar ValueError, y la excepción propia SinCambioDeSignoError hereda de ValueError. Una prueba recorre una lista con una Bisección y un Trapecio y verifica que ambos devuelven un Resultado.
- **Segregación de interfaces (ISP).** AlgoritmoNumerico solo declara lo que sus hijas realmente comparten; no se agregaron métodos «por las dudas».
- **Inversión de dependencias (DIP).** GeneradorCurva y los algoritmos dependen de la abstracción FuncionMatematica, no de una función concreta.
- **Sin duplicación de código.** El cálculo $v' = M \cdot v$ está escrito una sola vez en Transformacion.aplicar_punto, cada hija solo define su matriz. La validación del intervalo y de los valores de la función están en los métodos protegidos _validar_intervalo y _evaluar de AlgoritmoNumerico.
- **Encapsulamiento y diseño por contratos.** Los atributos se exponen mediante propiedades de solo lectura; los constructores verifican precondiciones e invariantes y lanzan excepciones cuando no pueden cumplirse (por ejemplo, un ResultadoBiseccion no puede tener la raíz fuera de [a, b]).

- **Relaciones.** Curva–Punto es una composición (cada curva tiene su propia copia de los puntos); las clases que reciben objetos como parámetro (generadores, transformaciones, algoritmos, visualizador, repositorio) se relacionan por dependencia.

Otras decisiones

- **Clase Punto:** tiene comportamiento propio (conversión a coordenadas homogéneas y distancia), que usan las transformaciones y el cálculo de longitud de la curva de Koch.
- **Clase Resultado:** la bisección debe informar cuatro datos (aproximación, iteraciones, convergencia y error); agruparlos en un objeto inmutable es más claro que una tupla y mantiene un tipo de retorno común para calcular().
- **JSON para la persistencia:** es texto legible e interoperable y, a diferencia de pickle, cargar un archivo no ejecuta código. Exige convertir la curva a datos simples y reconstruirla, que es justamente la responsabilidad del repositorio.
- **Visualizador y terminal:** según Graficación II se usa la API orientada a objetos de matplotlib (fig, ax). En uso normal se activa el modo interactivo (plt.ion()) para que la ventana se actualice sin bloquear el menú; las pruebas usan el backend Agg, que no abre ventanas.

iv. Algoritmo recursivo y algoritmos numéricos

Curva de Koch (recursivo)

GeneradorKoch construye la curva a partir del segmento de (0, 0) a (1, 0). El método recursivo koch(p1, p2, nivel) funciona así:

- **Caso base** (nivel = 0): el segmento no se subdivide y se devuelven sus extremos [p1, p2].
- **Reducción del problema** (nivel > 0): el segmento se divide en tercios con los puntos A (a 1/3) y B (a 2/3), y se calcula el vértice C del triángulo equilátero rotando 60° el vector AB alrededor de A. Así quedan **cuatro segmentos más cortos** (p1–A, A–C, C–B y B–p2), y sobre cada uno se hace una llamada recursiva con **nivel − 1**. Como cada llamada reduce el nivel en 1, siempre se llega al caso base.
- **Construcción de la solución:** las cuatro listas parciales se concatenan quitando el último punto de las tres primeras, porque es el primero de la siguiente, para no repetir puntos.
- **Límite de recursión:** NIVEL_MAXIMO = 6 (la consigna pide al menos 6). La profundidad de la pila es solo N, pero la cantidad de puntos crece como $4^N + 1$, por lo que limitar N evita un consumo de recursos desproporcionado.

```
def koch(self, p_p1, p_p2, p_nivel):
    if p_nivel == 0:                        # caso base
        return [p_p1, p_p2]
    # ... cálculo de los puntos A, B y C ...
    tramo1 = self.koch(p_p1, p_a, p_nivel - 1)   # 4 llamadas recursivas
    tramo2 = self.koch(p_a, p_c, p_nivel - 1)
    tramo3 = self.koch(p_c, p_b, p_nivel - 1)
    tramo4 = self.koch(p_b, p_p2, p_nivel - 1)
    return tramo1[:-1] + tramo2[:-1] + tramo3[:-1] + tramo4
```

Nivel N	0	1	2	3	4	5	6
Cantidad de puntos ($4^N + 1$)	2	5	17	65	257	1025	4097
Longitud ($(4/3)^N$)	1,0000	1,3333	1,7778	2,3704	3,1605	4,2140	5,6187

Tabla 3. Propiedades de la curva de Koch obtenidas con GeneradorKoch.

Método de bisección (RF4)

Clase Biseccion. El usuario establece la tolerancia, el número máximo de iteraciones o ambos, el que no indique toma un valor por defecto: 10^{-6} y 100.

1. Se valida que a < b y se evalúan f(a) y f(b). Si alguno de los extremos ya es raíz, se devuelve con 0 iteraciones.
2. Si f(a) y f(b) tienen el mismo signo se lanza SinCambioDeSignoError: no puede garantizarse una raíz, también cubre los casos con un número par de raíces.
3. Mientras el error sea mayor que la tolerancia y no se alcance el máximo de iteraciones, se toma el punto medio m y se conserva la mitad del intervalo donde hay cambio de signo. Si f(m) = 0, m es la raíz exacta.
4. La aproximación es el punto medio del último intervalo y el **error de estimación** es la mitad de su ancho: $(b - a)/2^{k+1}$ tras k iteraciones. Se informa si se alcanzó el criterio de convergencia (error ≤ tolerancia).

El bucle tiene dos condiciones de salida, por lo que siempre termina. Si la función devuelve un valor no finito (por ejemplo, un desborde de la exponencial) se lanza ValueError. Ejemplo: para $f(x) = x^2 - 2$ en [1, 2] con tolerancia 10^{-6} se obtiene la raíz 1,4142141 en 19 iteraciones, con error $9{,}5 \cdot 10^{-7}$, convergencia alcanzada. Para sen(x) en [3, 4] con solo 5 iteraciones se obtiene 3,140625 con error 0,0156, convergencia no alcanzada.

Método de los trapecios (RF5)

Clase Trapecio. El usuario establece la cantidad de subintervalos n (entero ≥ 1). Se calcula $h = (b - a)/n$, se suman $f(a)/2 + f(b)/2$ más los valores en los n − 1 nodos interiores y se multiplica por h; siempre se realizan exactamente n + 1 evaluaciones. El resultado en ResultadoIntegracion informa la aproximación y los parámetros utilizados a, b y n. La clase no usa matplotlib ni la consola, por lo que se prueba sin abrir ventanas, como exige RF5. Ejemplos: $\int_0^{\pi} \text{sen}(x)\,dx$ con n = 100 da 1,999836 (exacto: 2) y $\int_0^1 e^x\,dx$ con n = 50 da 1,718339 (exacto: e − 1 = 1,718282).

v. Estrategia de pruebas y resultados

Las pruebas se escribieron con **unittest** de biblioteca estándar y la cobertura se midió con **coverage.py**. Cada clase se probó a partir de su contrato, con casos de las cuatro categorías vistas en la Unidad 4: **habituales** datos representativos, **límites** extremos que ya son raíz, n = 1, nivel 0, **inválidos** intervalos con a ≥ b, tolerancias no positivas, tipos incorrectos y **excepcionales** funciones que devuelven valores no finitos.

- **Dobles de prueba (stub):** las pruebas de los algoritmos usan una función de prueba con un método evaluar(x), para no depender de otras clases del grupo (duck typing).
- **Sin ventanas:** las pruebas del Visualizador usan el backend Agg y verifican títulos, líneas, límites y leyendas a través de los ejes; los algoritmos y transformaciones se prueban independientemente del componente gráfico.
- **Ejecución:** python -m unittest discover -s tests -t . y, para la cobertura, python -m coverage run -m unittest discover -s tests -t . seguido de python -m coverage report. Se excluyeron de la medición los bloques if __name__ == "__main__", que son demostraciones.

Requisito de la consigna	Cómo se verifica	Archivo
Evaluación de las funciones	Curvas generadas con valores conocidos (x^2 en x = 0…4; seno; exponencial)	test_generadores.py
Transformaciones con puntos conocidos	Escalado, rotaciones de 90°, 180° y 360°, traslación, composición, curva original sin modificar	test_transformaciones.py, test_composicion_curva.py
Bisección con raíces conocidas	$\sqrt{2}$, raíz de $x^3 - x - 2$, extremos que son raíz, raíz en el punto medio, no convergencia, sin cambio de signo	test_algoritmos.py
Trapecio con resultado exacto conocido	$\int x^2$ (0,34375 con n = 4), $\int$ sen = 2, función lineal exacta, error ÷ 4 al duplicar n	test_algoritmos.py
Recursión en niveles pequeños	$4^N + 1$ puntos para N = 0…6, pico del nivel 1 en (0,5; $\sqrt{3}/6$), extremos conservados, niveles inválidos	test_generadores.py
Polimorfismo y resultados	Algoritmos intercambiables, resultados inmutables e invariantes	test_algoritmos.py, test_resultados.py
Visualización (RF2)	Título, ejes, curvas, raíz, intervalo, límites, escala igual y leyenda	test_visualizador.py
Persistencia	Guardar, cargar y no sobrescribir sin autorización	test_repositorio_curva.py

Tabla 4. Requisitos de prueba de la consigna y cómo se cubren.

Resultados: se ejecutaron **107 pruebas y todas pasaron**. La cobertura total es del **90 %**, por encima del 60 % requerido.

Paquete	Sentencias	Sin cubrir	Cobertura
algoritmos	82	0	100 %
resultados	67	0	100 %
generadores	71	2	97 %
visualizacion	82	8	90 %
transformaciones	91	10	89 %
funciones	92	14	85 %
persistencia	57	12	79 %
geometria	65	14	78 %
Total	607	60	90 %

Tabla 5. Cobertura por paquete (coverage.py).

PENDIENTE: volver a correr las pruebas y la cobertura con la versión final (incluyendo MenuConsola) y actualizar las Tablas 4 y 5. Se recomienda agregar pruebas directas de evaluar() para cada función matemática.

vi. Demostración funcional

Esta sección debería mostrar, con capturas de la aplicación, la generación, visualización, transformación, persistencia y análisis de al menos una curva. Secuencia propuesta:

1. Iniciar la aplicación (python main.py) y mostrar el menú principal.
2. Generar una curva a partir de una función (por ejemplo, $x^2 - 2$ en $[-2, 2]$ con 200 puntos) y mostrar la ventana con la curva.
3. Aplicar una transformación (por ejemplo, una rotación en radianes) y mostrar la curva original y la transformada.

4. Guardar la curva con un nombre corto, listarla y volver a cargarla (persistencia en data/).
5. Buscar una raíz con bisección y mostrar el resumen en la terminal y la raíz marcada en el gráfico.
6. Calcular la integral con el método de los trapecios y mostrar el resumen y los extremos del intervalo marcados.
7. Generar la curva de Koch de nivel 6 y ajustar la vista con escala igual en ambos ejes.

PENDIENTE: realizar la demostración con el menú terminado e insertar las capturas de la terminal y de la ventana en cada paso.

Bibliografía

- Booch, G. et al. Análisis y diseño orientado a objetos con aplicaciones. 2.ª ed. Addison Wesley Longman / Pearson Educación, 1996.
- PEP 8 – Style Guide for Python Code. https://peps.python.org/pep-0008/
- PEP 3119 – Introducing Abstract Base Classes. https://peps.python.org/pep-3119/
- Matplotlib. Interactive figures y Backends. https://matplotlib.org/stable/users/explain/figure/
- Material de la cátedra: Unidades 1 a 7 (modelo de objetos, UML, análisis y diseño, aspectos avanzados, manejo de excepciones, graficación, interfaces de usuario y algoritmos computacionales y numéricos).